

Optimizacija več-agentnega iskanja poti z uvedbo lokalnih vodij

Andraž Šošterič
andraz.sosteric@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Nejc Fras
nejc.fras@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

David Lipavec
david.lipavec@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Tadej Teršek
tadej.tersek@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

POVZETEK

V tem delu obravnavamo problem več-agentnega iskanja poti (MAPF) in predlagamo nov hibridni algoritem *Local Leaders*, ki združuje prednosti centraliziranega in decentraliziranega pristopa, za reševanje lifelong MAPF problemov. Algoritem agente dinamično razporeja v lokalne skupine, znotraj katerih vodja koordinira gibanje s prioriteto razrešitvijo konfliktov in algoritmom A*. Predlagani pristop primerjamo s centraliziranimi algoritmoma LaCAM in PBS, ter decentraliziranimi algoritmoma Follower in MATS-LP na standardiziranih zbirkah MovingAI in POGEMA. Rezultati kažejo, da *Local Leaders* dosega primerljiv ali celo nekoliko nižji SoC kot LaCAM (do 9 % manj pri 64 agentih), ob tem pa ohranja skoraj enako prepustnost kot MATS-LP na odprtih okoljih (97–101 %) in ga bistveno prekaša na skladiščnih scenarijih.

KLJUČNE BESEDE

centralizirano načrtovanje poti, distribuirano načrtovanje poti, usklajevanje agentov, lokalni vodje, več-agentno iskanje poti

1 UVOD

Problem večagentnega iskanja poti (ang. *Multi-Agent Path Finding*, MAPF), se v praksi pojavlja v številnih domenah, kot so robotizirana skladišča, avtonomna vozila, inteligentni transportni sistemi in drugi logistični sistemi. Gre za temeljni problem koordinacije več avtonomnih enot v skupnem prostoru, kjer mora vsak agent doseči svoj cilj brez trkov z drugimi agenti ali ovirami.

Cilj problema je načrtovati poti za množico agentov v diskretnem prostoru tako, da se izognemo medsebojnim konfliktom in hkrati optimiziramo določene metrike, kot sta skupni čas izvajanja (ang. *makespan*) ali vsota dolžin poti (ang. *sum of costs*) [10].

Obstoječe pristope delimo na centralizirane, katerih predstavnika sta LaCAM[6] in PBS[5], to so pristopi, ki načrtujejo poti za vse agente z globalnim pogledom na problem, in decentralizirane, kot sta Follower[1] in MATS-LP[8], kjer agenti odločitve sprejemajo lokalno brez centralnega koordinatorja. Vsaka skupina nosi znane kompromise: centralizirani pristopi zagotavljajo višjo kakovost rešitev, a slabše skalirajo z naraščajočim številom agentov;

decentralizirani dobro skalirajo, a pogosto na račun kakovosti. Ta kompromis je izhodišče za zasnovo našega algoritma *Local Leaders*.

Glavni prispevek tega dela je hibridni algoritem *Local Leaders* (lokalni vodje), pri katerem se agenti dinamično razporejajo v lokalne skupine, znotraj katerih izbrani vodja usklajuje premike in razrešuje konflikte po prioritetah. Hipoteza je, da tak pristop doseže boljše razmerje med kakovostjo rešitev in časom načrtovanja kot vsaka od obstoječih rešitev posebej, nižji SoC (vsoto stroškov) in višjo stopnjo uspeha kot decentralizirani pristopi, hkrati pa bistveno krajši čas načrtovanja kot centralizirani algoritmi.

Repozitorij z izvorno kodo projekta je na voljo na našem GitHub repozitoriju¹.

Stern et al. [10] definirajo problem več-agentnega iskanja poti (Multi-Agent Path Finding - MAPF) kot trojico

$$\langle G, s, t \rangle,$$

kjer je $G = (V, E)$ neusmerjen graf, pri čemer V predstavlja množico vozlišč, E pa množico povezav.

Podana je množica agentov $A = \{1, 2, \dots, k\}$. Funkcija

$$s : \{1, \dots, k\} \rightarrow V$$

preslika vsakega agenta v njegovo začetno vozlišče, funkcija

$$t : \{1, \dots, k\} \rightarrow V$$

pa v njegovo ciljno vozlišče.

Naj bo $\pi_i[t] \in V$ položaj agenta i v diskretnem času $t \in \mathbb{N}_{\geq 0}$. Pri vsakem koraku t se agent i lahko premakne v sosednje vozlišče ali ostane na trenutnem, kar formalno zapišemo kot

$$\pi_i[t+1] \in \text{neigh}(\pi_i[t]) \cup \{\pi_i[t]\},$$

kjer je $\text{neigh}(v)$ množica vozlišč, sosednjih vozlišču $v \in V$ [6].

Okamura [6] definira vrste konfliktov, ki se lahko pojavijo med načrtovanjem poti, in jih mora algoritem upoštevati. Agent se mora izogibati naslednjim vrstam konfliktov:

¹<https://github.com/Asos75/MultiAgent>

- **Vozliščni konflikt:** Do vozliščnega konflikta (ang. *vertex conflict*) pride, če nameravata π_i in π_j zasedeti isto vozlišče v istem trenutku, torej obstaja časovni korak t , da velja $\pi_i[t] = \pi_j[t]$. [10]
- **Zamenjalni konflikt:** Do zamenjalnega konflikta (ang. *swap conflict*) pride, če nameravata π_i in π_j zamenjati svoji poziciji v istem trenutku, torej obstaja časovni korak t , da velja $\pi_i[t] = \pi_j[t+1] \wedge \pi_i[t+1] = \pi_j[t]$ [10].

Rešitev MAPF za instance, omenjene zgoraj, je množica brezkonfliktnih poti $\{\pi_1, \dots, \pi_k\}$ takih, da obstaja časovni korak $T \in \mathbb{N}_{\geq 0}$, kjer agent doseže cilj. [6]

Članek je organiziran na naslednji način: v razdelku *Sorodna dela* povzamemo izbrane centralizirane in decentralizirane pristope MAPF, v razdelku *Uvedba lokalnih vodij* predstavimo naš pristop *Local Leaders*, v razdelku *Eksperiment in analiza rezultatov* opišemo eksperimentalno okolje in metrike ter analiziramo rezultate, na koncu pa v *Zaključku* strnemo ugotovitve in predlagamo nadaljnje delo.

2. SORODNA DELA

V tem razdelku izpostavimo ključna dela s področja več-agentnega iskanja poti, ki so motivirala zasnovo algoritma *Local Leaders* ali zagotavljajo referenčne točke za njegovo vrednotenje.

EECBS [4] je nadgradnja klasičnega CBS, ki ohrani dvonivojsko iskanje, hkrati pa z *Explicit Estimation Search* in uteževanjem omogoča nadzorovano suboptimalnost (uporabnik nastavi kompromis med kakovostjo in časom). Avtorji poročajo o bistveno boljši skalabilnosti na večjih instancah ob majhni, dokazljivo omejeni degradaciji kakovosti rešitve. *EECBS* izpostavi kompromis »hitrost vs. kakovost« pri centraliziranih metodah in utemeljuje, zakaj smo v zasnovi *Local Leaders* ciljali na suboptimalno, a hitro lokalno načrtovanje; prav tako pojasnjuje, zakaj v evalvaciji poleg kakovosti (SoC/makespan) spremljamo tudi čas izvajanja in skalabilnost.

CBSH2 [3] izboljša CBS z močnejšimi heuristikami in tehniko *disjoint splitting*, ki z uvedbo pozitivnih omejitev zmanjša podvajanje iskanja in s tem velikost iskalnega drevesa. Rezultat je hitrejše reševanje in boljša skalabilnost ob ohranjeni rešitveni kakovosti tipične za CBS-družino. *CBSH2* pokaže, da so napredki centraliziranih metod pogosto posledica boljšega upravljanja konfliktov; ta ugotovitev neposredno motivira dvofazno razrešitev konfliktov v *Local Leaders*, ki je zasnovana kot lokalna analogija te ideje, torej brez globalnega iskalnega drevesa.

Follower [1] je decentraliziran pristop za neprekinjeni (ang. *lifelong*) MAPF, ki kombinira (i) heuristični planer (npr. A^* nad statičnimi ovirami), ki generira »razpršene« poti z namenom zmanjšanja zasičenosti ozkih grl, in (ii) naučeno politiko sledenja, ki lokalno reagira na druge agente in se izogiba trkom. Izvajanje je povsem lokalno (brez centralnega koordinatorja), zato metoda dobro skalira, vendar je kakovost poti odvisna od lokalnih informacij in stabilnosti naučene politike. *Follower* je ena od referenčnih baseline metod za vrednotenje *Local Leaders*: ker oba delujeta lokalno brez centralnega koordinatorja, primerjava razkrije, koliko koordinacija z vodji doda v primerjavi z naučeno politiko sledenja, brez potrebe po učenju.

Decentralized Monte Carlo Tree Search for Partially Observable Multi-agent Pathfinding [9] predlaga decentralizirano reševanje neprekinjenega (ang. *lifelong*) MAPF pri delni opazljivosti: vsak

agent iz lokalnih opazovanj zgradi intrinzični (lokalni) Markovski odločitveni proces (ang. *Markov Decision Process*, MDP) in na njem izvaja Monte Carlo drevesno iskanje (ang. *Monte Carlo Tree Search*, MCTS), ki ga dodatno usmerja nevronska politika. S simulacijami v drevesu agenti ocenjujejo kratkoročne interakcije z bližnjimi agenti, koordinacija pa nastaja implicitno (brez eksplicitne komunikacije ali centralnega koordinatorja), kar vodi do dobre skalabilnosti. Delo kaže, da koordinacija z lokalnim pogledom vnaprej (MCTS) izboljša prepustnost pri delni opazljivosti; *Local Leaders* dosega primerljivo koordinacijo z eksplicitno hierarhijo vodij, brez računsko zahtevnega drevesnega iskanja.

PRIMAL2 [2] je zgodnje delo, ki pokaže, da je mogoče MAPF reševati z decentraliziranimi politikami na osnovi lokalnih opazovanj, naučenimi s kombinacijo posnemovalnega učenja (ang. *imitation learning*) iz centraliziranih planerjev in okrepitevenega učenja (ang. *reinforcement learning*). Pristop se zelo dobro skalira, vendar pogosto žrtvuje optimalnost in lahko povzroča neuskkljenosti v gostih scenarijih. *PRIMAL2* potrjuje, da je suboptimalnost SoC pri učenih decentraliziranih metodah pričakovan kompromis, to odpira prostor za pristop z eksplicitno koordinacijo vodij, ki ne zahteva učenja in si prizadeva zmanjšati ta kompromis.

SCRIMP [11] uvede eksplicitno komunikacijo med agenti z uporabo arhitekture Transformer in pozornostnega mehanizma, kar omogoča učinkovitejšo koordinacijo tudi pri zelo omejenem vidnem polju. S tem pristop preseže omejitve povsem nekomunikacijskih lokalnih politik, hkrati pa ohrani decentralizirano izvajanje in dobro skalabilnost. *SCRIMP* kaže, da dodaten koordinacijski signal, bodisi eksplicitna komunikacija bodisi pozornostni mehanizem, bistveno izboljšata prepustnost. *Local Leaders* sledi tej ugotovitvi z lažjim mehanizmom koordinacije prek vodij, ki ne zahteva niti predhodnega učenja niti eksplicitne komunikacije med agenti.

3. UVEDBA LOKALNIH VODIJ

Glavni prispevek tega dela je predlagana nadgradnja obstoječih pristopov, ki uvaja koncept *lokalnih vodij*. Gre za algoritem *Local Leaders*, ki združuje prednosti centraliziranega in decentraliziranega pristopa. Algoritem deluje lokalno, na ravni agenta, brez vnaprejšnjega načrtovanja in deluje v petih korakih (Slika 1).

1. *Oblikovanje skupin*. Na začetku vsakega koraka oz. vsakih $t_{\text{regroup}} = 5$ korakov se agenti razdelijo v disjunktne skupine. Algoritem obdeluje agente v naraščajočem vrstnem redu ID-jev (od 0 do $k - 1$), ki so agentom dodeljeni na začetku algoritma in se v nadalje ne spreminjajo. Za vsakega agenta i se med vsemi že identificiranimi vodji (to so agenti $j < i$, ki so v tej iteraciji postali vodje) poišče najbližji po Manhattanski razdalji znotraj polmera $r_{\text{leader}} = 17$ (vidno polje vodje). Če je tak vodja najden, se agent i pridruži njegovi skupini; sicer pa agent i sam postane vodja. Agenti, ki niso vodja, se imenujejo *sledilci*. Vodja opazuje člane skupine v polmeru r_{leader} , sledilci pa zaznavajo sosednje agente v polmeru $r_{\text{agent}} = 11$ (vidno polje agenta). Ker se agenti obdelujejo v strogo naraščajočem vrstnem redu ID-jev, je particija enolično določena in brez dvumnosti.

2. *Izračun željenih celic (A^*)*. Vsak agent neodvisno izvede algoritem A^* od svojega položaja do cilja, pri čimer upošteva le statične ovire in ignorira položaje ostalih agentov. Rezultat je *željena celica*,

to je celica, v katero se agent želi premakniti v tem koraku. Agent se lahko prav tako odloči, da ostane na istem mestu (brez premika), kar je vedno veljavna akcija.

Uvajamo **prag pobega** t_{esc} (ang. *escape threshold*): to je celo število, ki določa, po koliko zaporednih korakov brez premika agent preide v *način pobega*. Kadar je agentov števec zastoja (število zaporednih korakov, v katerih se ni premaknil) enak ali večji od t_{esc} , agent namesto A^* izbere naključno prosto sosednjo celico. Kadar nobena sosednja celica ni prosta, agent ostane na mestu, torej je njegova *željena celica* je enaka trenutnemu položaju. Opozorimo, da agent lahko ostane na mestu tudi zunaj načina pobega, kadar razrešitev konfliktov (korak 3) ne pusti nobene boljše alternative. Parameter: $t_{esc} = 1$, torej agent vstopi v način pobega že po enem koraku brez premika.

3. *Razrešitev konfliktov v dveh fazah.* Razrešitev poteka v dveh eksplicitnih fazah, ki ločita konflikte *znotraj* skupin od konfliktov *med* skupinami.

Faza A – znotrajskupinska razrešitev

Vsak vodja neodvisno razrešuje konflikte znotraj svoje skupine. Obdelava članov poteka v naslednjem vrstnem redu:

- (1) agenti v načinu pobega,
- (2) preostali člani, urejeni po bližini do cilja,
- (3) agenti z nižjim ID-jem.

Za vsakega člana skupine vodja preveri tako *vozliščne* (dva agenta želita v isto celico), kot tudi *zamenjalne* konflikte (agenti menjujeta poziciji). Preverjanje se ponovi za vsakega člana posebej, kar pomeni, da vsak agent preverja konflikte z vsemi že potrjenimi premiki predhodnih članov. Agent, ki pride na vrsto pozneje, se ob konfliktu preusmeri v najboljšo prosto alternativo (celico, ki ni rezervirana in je čim bližje prvotni željeni celici). Kadar take alternative ni, agent ostane na mestu. Rezultat tega koraka so željeni položaji vsakega agenta. Ti položaji so začasni, ker se v fazi B še preverijo konflikti med skupinami in se lahko nekateri agenti dodatno preusmerijo.

Faza B – medskupinska razrešitev

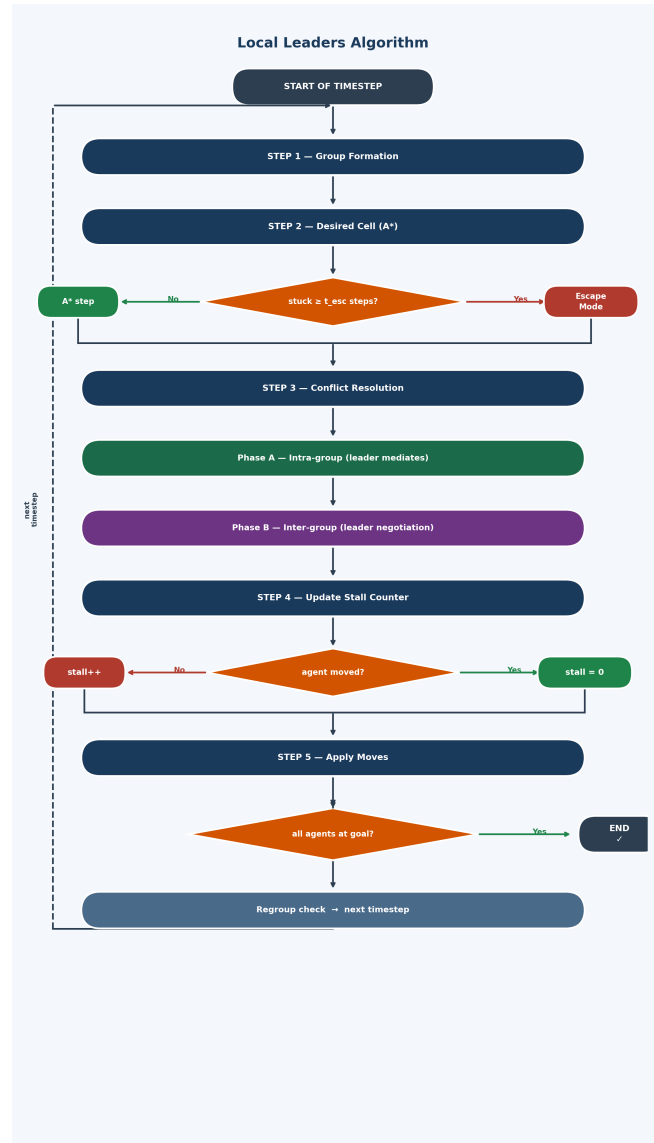
Za razrešitev konfliktov med skupinami uvajamo *karto premikov*, to je skupna globalna podatkovna struktura, ki hrani, katero celico je kateri agent že rezerviral za tekoči korak. Pred vpisom skupin v karto premikov se določi naslednja prioriteta:

- (1) agenti v načinu pobega, ne glede na skupino,
- (2) skupina z večjim številom običanih agentov,
- (3) skupina, katere vodja je bližje svojemu cilju,
- (4) skupina z nižjim ID-jem vodje.

Skladno s to prioriteto se skupine zavežejo v karto premikov. Za vsakega agenta se zaporedno preverita vozliščni in zamenjalni konflikt z vsemi že vpisanimi agenti, tako kot v fazi A. Ko agent iz nižjeprioritene skupine zahteva celico, ki jo je višjeprioritena skupina že zavzela, se agent preusmeri v najboljšo prosto alternativo ali, kadar te ni, ostane na mestu.

4. *Posodobitev števca zastoja.* Agent, ki se ni premaknil, poveča števec zastoja, medtem ko ga agent, ki se je premaknil, ponastavi na nič.

5. *Izvedba premikov.* Vsi agenti hkrati izvedejo svoje potrjene premike, torej se premaknejo v svoje željene celice, ki so bile potrjene v fazi B.



Slika 1: Diagram poteka algoritma Local Leaders: dinamično oblikovanje skupin, določitev vodje, A^* načrtovanje in dvofazna razrešitev konfliktov (znotrajskupinsko reševanje konfliktov, nato medskupinsko pogajanje vodij).

6. *Implementacija.* Algoritem je implementiran v Pythonu za integracijo z okoljem POGEMA in MovingAI, ter v C++ (prek pybind11) za hitrejše izvajanje.

Lokalni vodja nima globalnega pogleda nad celotnim okoljem, temveč razpolaga le z informacijami o agentih znotraj svoje skupine in njihove neposredne okolice. Na ta način se bistveno zmanjša kompleksnost načrtovanja v primerjavi s centraliziranimi algoritmi, hkrati pa se ohrani višja stopnja koordinacije kot pri popolnoma decentraliziranih pristopih.

4 EKSPERIMENT

V tem poglavju bomo predstavili metodologijo vrednotenja in primerjanja izbranih algoritmov za problem večagentnega iskanja poti. Cilj eksperimentalnega dela je primerjati centralizirane pristope, kot sta LaCAM in PBS, z decentraliziranimi pristopi Follower in MATS-LP ter ovrednotiti predlagani pristop z lokalnimi vodji.

4.1 Eksperimentalno okolje

Za evalvacijo uporabljamo knjižnico POGEMA [7], ki ponuja standardizirano platformo za primerjavo MAPF algoritmov v okoljih z nepopolno informacijo, z generatorjem instanc, enotnimi metrikami in protokolom izvajanja. To kombiniramo z MovingAI [10] zbirko zemljevidov, ki zagotavlja reprezentativne testne scenarije.

Eksperimenti so razdeljeni v dve skupini glede na način delovanja: na zbirkah MovingAI izvajamo *enkratne* (ang. *one-shot*) eksperimente, kjer vsak agent po dosegu cilja ustavi (`on_target=finish`), ter merimo SoC, makespan in stopnjo uspeha. Na zbirkah POGEMA pa izvajamo *neprekinjene* (ang. *lifelong*) eksperimente, kjer agent po dosegu cilja prejme novo nalogo (`on_target=restart`); prepustnost merimo z metriko `LifeLongAverageThroughputMetric` iz knjižnice POGEMA, kar zagotavlja neposredno primerljivost med algoritmi Local Leaders, MATS-LP in Follower.

4.2 Metrike

Vsota stroškov (SoC; ang. *sum of costs*) je vsota časovnih korakov, ki jih vsi agenti porabijo za dosego svojih ciljnih vozlišč [10]:

$$\text{SoC} = \sum_{i=1}^k |\pi_i|,$$

kjer je $|\pi_i|$ dolžina poti agenta i . Nižja vrednost pomeni učinkovitejšo rešitev.

SoC je ena izmed najpogostejše uporabljenih metrik v MAPF literaturi [10], kar omogoča neposredno primerjavo z obstoječimi deli. Metrika meri skupni strošek kot vsoto dolžin poti vseh agentov in s tem neposredno zajame celotno učinkovitost sistema.

Metrika je še posebej primerna za enkratne (ang. *one-shot*) scenarije, saj minimizacija SoC ustreza cilju minimizacije skupne porabe časa oziroma virov. Hkrati kaznuje neučinkovite poti posameznih agentov, kar spodbuja iskanje globalno učinkovitih rešitev. SoC bomo zato uporabili za vrednotenje algoritmov LaCAM in PBS na zbirki MovingAI.

Makespan je čas, ki ga porabi najdaljša pot med vsemi agenti, torej maksimum nad vsemi $|\pi_i|$ [10]:

$$\text{makespan} = \max_{i \in \{1, \dots, k\}} |\pi_i|.$$

Metrika odraža skupno trajanje reševanja instance in je primerna za scenarije, kjer je ključen vzporedni zaključek vseh agentov. Makespan je uveljavljena metrika v MAPF literaturi [10] in je še posebej relevantna v praktičnih aplikacijah, kjer je ključen vzporedni zaključek vseh agentov.

Makespan je pri primerjavi centraliziranih algoritmov nujna dopolnitev k SoC, saj sama vsota stroškov namreč ne razkrije, ali posamezen agent morda bistveno zavlaja celotno skupino. Z vključitvijo obeh metrik tako dobimo celovitejšo in bolj zanesljivo oceno učinkovitosti algoritma v realnih pogojih.

Stopnja uspeha (ang. *success rate*) meri delež instanc, pri katerih vsi agenti uspešno dosežejo svoje cilje znotraj predvidenega časovne omejitve [7]:

$$SR = \frac{1}{N} \sum_{n=1}^N \mathbf{1}[\forall i \in A, \exists t \leq T_{\max} : \pi_i[t] = t(i)],$$

kjer je N skupno število testnih instanc, $T_{\max}$ časovna omejitev, $t(i)$ ciljno vozlišče agenta i in $\mathbf{1}[\cdot]$ indikatorska funkcija (1 kadar je pogoj izpolnjen, 0 sicer).

Stopnja uspeha je primarna metrika platforme POGEMA in skupaj s prepustnostjo (ang. *throughput*) tvori osnovo za vrednotenje na tej platformi [7]. Zasnovana je posebej za vrednotenje decentraliziranih metod v delno opazljivih okoljih. Metrika je za naš eksperiment ključna, saj za razliko od SoC in makespana meri tudi robustnost algoritma oziroma njegovo sposobnost, da instanco v celoti reši kljub omejenim lokalnim opazovanjem.

Prepustnost (ang. *throughput*) meri povprečno število agentov, ki na časovni korak dosežejo svoja ciljna vozlišča [7]:

$$\text{throughput} = \frac{1}{T} \sum_{t=1}^T |\{i \in A : \pi_i[t] = t(i)\}|,$$

kjer je T skupno število časovnih korakov epizode in $t(i)$ ciljno vozlišče agenta i . Skupaj s stopnjo uspeha tvori osnovo za vrednotenje na platformi POGEMA [7]. Metrika je še posebej primerna za neprekinjene (ang. *lifelong*) oziroma zvezne scenarije, kjer agenti po dosegu cilja prejmejo novo nalogo, saj odraža dolgotrajno zmogljivost sistema pri visokih obremenitvah. Za razliko od makespana, ki meri trajanje posamezne instance, prepustnost zajame sposobnost sistema za vzdrževanje učinkovitega delovanja skozi daljše časovno obdobje.

4.2.1 Testne instance. Po začetni validaciji implementacij smo zasnovali lasten eksperiment, ki primerja centralizirane, decentralizirane in hibridni pristop z lokalnimi vodji. Namen eksperimenta je preveriti ali lahko uvedba lokalnih vodij izboljša razmerje med kakovostjo rešitev in časom načrtovanja.

Eksperimente izvajamo na standardiziranih zemljevidih iz zbirke MovingAI in POGEMA, na različnih velikostih zemljevidov, gostotah agentov in scenarijih. Tu so najbolj zanimivi scenariji z visoko gostoto agentov, kjer se razlike med rešitvami najbolj pokažejo.

Uporabljamo tri glavne tipe okolij:

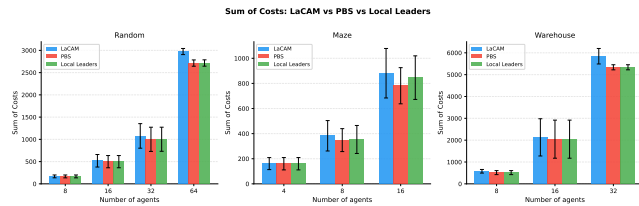
- **Odperte mreže** (ang. *random grids*) – okolja z malo ovirami, kjer je večina konfliktov posledica visoke gostote agentov.
- **Labirinti** (ang. *mazes*) – okolja z ozkimi prehodi in številnimi lokalnimi ozkimi grli, kjer je koordinacija agentov bistveno zahtevnejša.
- **Skladiščni scenariji** (ang. *warehouse maps*) – realistične postavitev, podobne robotiziranim skladiščem, kjer se agenti pogosto srečujejo na križiščih in v ozkih hodnikih.

Za vsako vrsto okolja izvajamo eksperimente pri različnih številih agentov (od 20 do 1000 agentov, odvisno od velikosti zemljevida), z različnimi konfiguracijami. Vsaka konfiguracija se izvede večkrat z različnimi scenariji, ki vključujejo začetne in ciljne pozicije agentov, rezultati pa se povprečijo, da zmanjšamo vpliv posameznih naključnih primerov.

5 ANALIZA REZULTATOV

5.1 Vsota stroškov (SoC)

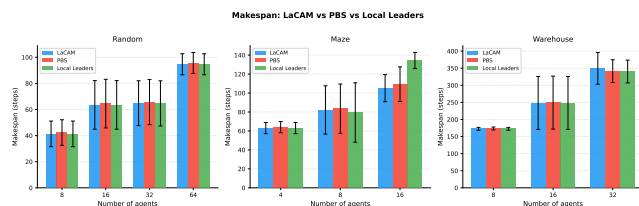
Slika 2 prikazuje vsoto stroškov za LaCAM, PBS in Local Leaders na treh tipih zemljevidov pri različnih številih agentov.



Slika 2: Vsota stroškov (SoC): LaCAM, PBS in Local Leaders na naključnih, labirintnih in skladiščnih zemljevidih. Napake kažejo standardno deviacijo čez 3 semena.

Ključna ugotovitev je, da tako PBS kot Local Leaders dosledno dosegata nižji SoC kot LaCAM pri vseh tipih zemljevidov. Na naključnih zemljevidih pri 8 agentih so vrednosti skoraj identične (LaCAM: 168, PBS: 167, LL: 167), pri 64 agentih pa Local Leaders doseže SoC 2716, PBS pa 2715, medtem ko LaCAM doseže 2974, kar je $\approx 9\%$ manj za oba. Na skladiščnih zemljevidih je prihranek pri 32 agentih $\approx 9\%$ (PBS: 5337, LL: 5338 vs. LaCAM: 5844). Na labirintnih zemljevidih PBS doseže SoC 782 pri 16 agentih (vs. LaCAM 881, $\approx 11\%$ manj), Local Leaders pa 846 ($\approx 4\%$ manj, le pri uspešnih instancah). Rezultat je nekoliko presenetljiv, saj LaCAM minimizira SoC globalno, medtem ko PBS in Local Leaders delujeta lokalno oziroma s prioriteto razrešitvijo, vzrok tiči v tem, da algoritma Local Leaders in PBS iščeta najkrajše poti, medtem ko LaCAM išče zagotovljeno rešitev.

5.2 Makespan

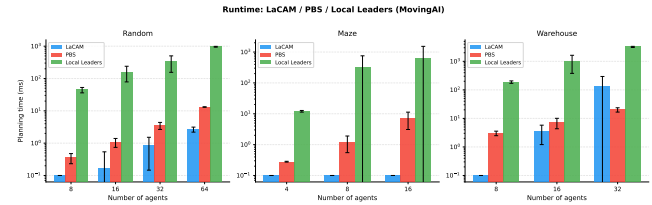


Slika 3: Makespan: LaCAM, PBS in Local Leaders. Na naključnih in skladiščnih zemljevidih so vsi trije algoritmi praktično identični; na labirintih PBS doseže makespan 109 pri 16 agentih (vs. LaCAM 105), Local Leaders pa 134, kar je posledica delno neuspešnih instanc.

Slika 3 kaže, da je makespan vseh treh algoritmov na naključnih in skladiščnih zemljevidih skoraj enak. Na naključnih zemljevidih z 8 agenti so vrednosti identične (LaCAM: 41, PBS: 42, LL: 41), pri 64 agentih pa vsi dosežejo makespan 95–96. Na labirintih PBS ohranja 100 % uspešnost in doseže makespan 109 pri 16 agentih (LaCAM: 105), torej le minimalno višje. Izjema je Local Leaders: pri 16 agentih doseže makespan 134, ker uspešno reši le 50 % instanc (povprečje

zajema le uspešne poskuse). PBS tako zapolni vrzel, ki jo Local Leaders pušča v scenarijih z ozkimi hodniki.

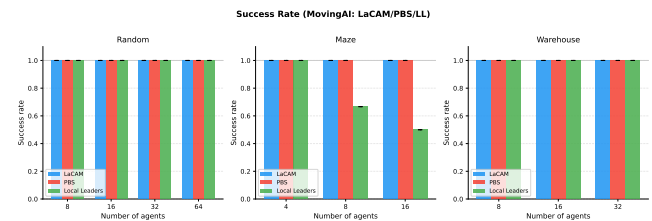
5.3 Čas izvajanja



Slika 4: Čas načrtovanja LaCAM, PBS in Local Leaders na MovingAI v logaritemski skali (ms).

Slika 4 razkrije eno ključnih razlik med pristopi. Na MovingAI LaCAM porabi manj kot 1 ms za manjše instance in do 128 ms za 32 agentov na skladiščnih zemljevidih. PBS je le malce počasnejši: 0,3–20 ms za vse scenarije, torej blizu LaCAM, a bistveno hitrejši od Local Leaders, ki deluje sproti (ang. *online*; A* za vsakega agenta pri vsakem koraku) in porabi 44–961 ms za ključne ter 188–3203 ms za skladiščne scenarije. PBS tako ponuja najboljše razmerje med kakovostjo rešitve in hitrostjo med vsemi tremi enkratnimi (ang. *one-shot*) algoritmi.

5.4 Stopnja uspeha in prepustnost na POGEMA



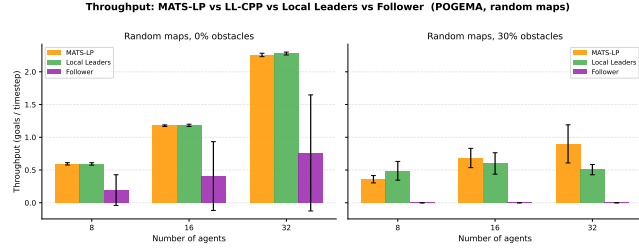
Slika 5: Levo: stopnja uspeha LaCAM, PBS in Local Leaders na MovingAI enkratnih (ang. *one-shot*) scenarijih. Desno: prepustnost v neprekinjenih (ang. *lifelong*) scenarijih MATS-LP, Local Leaders in Follower na POGEMA (naključni zemljevidi, 0 % ovir).

Slika 5 prikazuje dve dimenziji primerjave. V levem delu so stopnje uspeha na MovingAI: na naključnih in skladiščnih zemljevidih vsi trije algoritmi dosežejo 100 % pri vseh gostotah agentov. Na labirintnih zemljevidih LaCAM in PBS vedno uspeša, medtem ko Local Leaders dosega le 67 % (8 agentov) oziroma 50 % (16 agentov). PBS tukaj izrazito prednjači, ozkih hodnikov ne rešuje reaktivno, ampak z optimalnim iskanjem, ki zagotovi pot za vsakega agenta. To je primarna prednost PBS pred Local Leaders v gostih labirintnih scenarijih.

Desni panel prikazuje prepustnost v neprekinjenih (ang. *lifelong*) scenarijih (cilji/korak) na POGEMA. Rezultati prikazujejo C++ implementacijo Local Leaders, ki je po prepustnosti primerljiva s Python različico, a bistveno hitrejša pri izvedbi. MATS-LP doseže

0,61 (8 agentov), 1,18 (16) in 2,26 (32 agentov). Local Leaders je tik zadaj: 0,59, 1,18 in 2,28, kar je 97–101 % prepustnosti MATS-LP. Follower dosega nižje vrednosti (0,36, 0,76, 1,41).

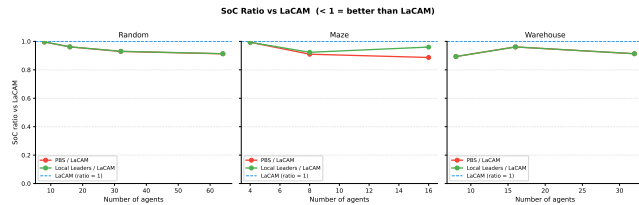
5.5 Primerjava prepustnosti pri 30 % ovirah



Slika 6: prepustnost v neprekinjenih (ang. *lifelong*) scenarijih MATS-LP, Local Leaders in Follower na naključnih zemljevidih pri 0 % (levo) in 30 % (desno) gostoti ovir. Rezultati Local Leaders so iz C++ implementacije. Pri 30 % ovirah Local Leaders ostaja konkurenčen pri manjšem številu agentov, a zaostaja pri 32 agentih.

Slika 6 kaže, kako gostota ovir vpliva na vsak algoritem. Pri 0 % ovirah so vse tri krivulje blizu skupaj in dobro skalirajo z naraščajočim številom agentov. Pri 30 % ovirah se slika nekoliko spremeni: MATS-LP ohranja razumno prepustnost (0,36–0,90), Follower je pri 32 agentih primerljiv z MATS-LP (0,94). Local Leaders s posodobitvijo parametrov ($t_{esc} = 1$) in dvofazno razrešitvijo konfliktov bistveno izboljša prepustnost pri gostih ovirah: pri 8 agentih doseže 0,49 (vs. MATS-LP 0,36), pri 16 agentih 0,60, pri 32 agentih pa 0,51 (vs. MATS-LP 0,90). Agresivnejši mehanizem pobega in posredovanje vodje znotraj skupin učinkoviteje razrešujeta ciklične zastoje v ozkih hodnikih, čeprav je pri visokem številu agentov MATS-LP še vedno boljši.

5.6 SoC overhead razmerje

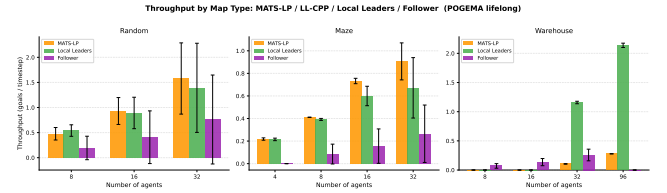


Slika 7: Razmerje SoC (PBS / LaCAM in Local Leaders / LaCAM) po tipu zemljevida. Vrednost pod 1 pomeni nižji SoC kot LaCAM.

Slika 7 prikazuje razmerje SoC za oba algoritma glede na LaCAM. Oba, PBS in Local Leaders, pri vseh tipih zemljevidov in gostotah agentov dosežeta nižji SoC kot LaCAM (razmerje < 1,0). Na naključnih zemljevidih razmerji PBS in Local Leaders sledita podobni krivulji, ki pada z naraščajočim številom agentov: od $\approx 0,994$ (8 agentov) do $\approx 0,913$ (64 agentov), vrednosti so tako podobne, da se

črti praktično prekrivata. Na skladiščnih zemljevidih je razmerje za oba med 0,888 (8 agentov) in 0,913 (32 agentov). Na labirintnih zemljevidih PBS doseže razmerje 0,888 pri 16 agentih, Local Leaders pa 0,960 (le pri uspešnih instancah), PBS je tu boljši, ker rešuje vse instance.

5.7 prepustnost po tipu zemljevida



Slika 8: prepustnost v neprekinjenih (ang. *lifelong*) scenarijih MATS-LP, Local Leaders in Follower po tipu zemljevida (naključni, labirint, skladišče). Na skladiščnih zemljevidih Local Leaders dosega do 10× višjo prepustnost kot MATS-LP.

Slika 8 primerja prepustnost v neprekinjenih (ang. *lifelong*) scenarijih MATS-LP, Local Leaders in Follower na vseh treh tipih zemljevidov. Na naključnih zemljevidih so si algoritmi po prepustnosti blizu. Na labirintih pa vodi MATS-LP (0,906 pri 32 agentih), Local Leaders pa doseže 0,672 (pri 32 agentih), kar je boljše od Followerja. Na skladiščnih zemljevidih se pa vlogi zamenjata: Local Leaders doseže 1,16 (32 agentov) oziroma 2,14 (96 agentov), MATS-LP pa le 0,11 oziroma 0,28, to je ≈ 8 –10-kratna prednost. Follower na skladiščnih zemljevidih ne uspe zaključiti epizod. Vzrok je v tem, da MCTS v širokih hodnikih postane računsko izjemno zahteven (do 1240 s na epizodo), medtem ko Local Leaders s C++ implementacijo isto epizodo opravi v pod 2 s.

5.8 Povzetek rezultatov

Tabela 1: Povzetek primerjave po metrikah. ++ zelo dobro, + dobro, ~ primerljivo, - slabo, -- zelo slabo / N/A.

Metrika	LaCAM	PBS	LL	MATS-LP	Follower
SoC (enkratni)	~	+	+	-	-
Makespan (enkratni)	+	+	+	-	-
Čas načrtov. (MovingAI)	++	+	-	-	-
Uspešnost (naključni)	++	++	++	-	-
Uspešnost (labirinti)	++	++	-	-	-
prepustnost (0 % ovire)	-	-	~	+	~
prepustnost (30 % ovire)	-	-	~	+	~
prepustnost (skladišče)	-	-	++	-	-
Čas epizode (POGEMA)	-	-	++	-	+

Preglednica 1 povzema primerjavo vseh petih algoritmov. PBS se izkaže kot izredno kompetenten centraliziran algoritem: na vseh tipih zemljevidov doseže 100 % uspešnost (vključno z labirinti, kjer Local Leaders zataji pri 33–50 %), z SoC, ki je ≈ 9 –11 % nižji kot pri LaCAM, in časom načrtovanja 0,3–20 ms. S tem zapolni vrzel, ki jo

pušča Local Leaders v gostih labirintnih scenarijih, brez kompromisa pri hitrosti.

Local Leaders se izkaže kot učinkovit pristop: v neprekinjenih (ang. *lifelong*) scenarijih dosega 97–101 % prepustnosti MATS-LP na odprtih naključnih okoljih, na skladiščnih okoljih pa ga $\approx$ 8–10× prekaša. Dvofazna razrešitev konfliktov z vodji bistveno izboljša prepustnost v labirintih (+23 % pri 32 agentih) in pri 30 % ovirah (0,51 vs. 0,90 MATS-LP pri 32 agentih). Follower, kot naučena politika, leži med njima, a ne uspe zaključiti epizod na skladiščnih zemljevidih.

6 ZAKLJUČEK

V tem delu smo predstavili algoritem *Local Leaders*, ki je naš pristop za reševanje problema večagentnega iskanja poti. Ta temelji na dinamičnem oblikovanju hierarhije lokalnih skupin z vodji, neodvisnem načrtovanju z algoritmom A^* in reševanju konfliktov z uporabo prioritet, s tem da se načrtovanje izvaja sproti.

Naši eksperimenti na zbirkah MovingAI in POGEMA so pokazali:

- **Kakovost rešitev:** Local Leaders premaga LaCAM v metriki Soc, (do 9 % manj) in dosega primerljiv makespan na naključnih in skladiščnih, kar je za lokalni algoritem presemetljivo.
- **prepustnost:** Na odprtih (0 % ovir) naključnih okoljih dosega Local Leaders 97–101 % prepustnosti MATS-LP. Na labirintih doseže 0,672 pri 32 agentih (+23 % v primerjavi z osnovno različico). Na skladiščnih okoljih prekaša MATS-LP za 8–10-kratnik (2,14 vs. 0,28 ciljev/korak pri 96 agentih). Pri 30 % ovirah dosega 0,49–0,60 ciljev/korak, kar je bistveno boljše od osnove (bila 0,14–0,28).
- **Slabosti:** Algoritem pri 32 agentih in 30 % ovirah zaostaja za MATS-LP (0,51 vs. 0,90), v labirintnih okoljih pa PBS ponuja zanesljivejšo alternativo z zagotovljeno 100 % uspešnostjo.

Našo hipotezo, da hibridni pristop z lokalnimi vodji doseže boljše razmerje med časom načrtovanja in kakovostjo pridobljenih rešitev kot obstoječi centralizirani ali decentralizirani pristopi, potrjujemo za odprta, gosta in skladiščna okolja. Dvofazna razrešitev konfliktov z vodji se je izkazala kot soliden pristop. Za scenarije z ozkimi hodniki pa bo potrebno razviti zanesljivejši mehanizem razrešitve zastojev.

LITERATURA

- [1] Anton Andreychuk, Konstantin Yakovlev, Alexey Skrynnik, and Aleksandr Panov. 2024. Follower: Learning to Solve Decentralized Lifelong Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press.
- [2] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. 2021. PRIMAL₂: Pathfinding via Reinforcement and Imitation Multi-Agent Learning – Lifelong. *IEEE Robotics and Automation Letters* 6, 2 (2021), 2666–2673. <https://doi.org/10.1109/LRA.2021.3062803>
- [3] Jiaoyang Li, Daniel Harabor, Peter J. Stuckey, Ariel Felner, Hang Ma, and Sven Koenig. 2019. Disjoint Splitting for Multi-Agent Path Finding with Conflict-Based Search. 29 (Jul. 2019), 279–283. <https://doi.org/10.1609/icaps.v29i1.3487>
- [4] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. 2021. EECBS: A Bounded-Suboptimal Search for Multi-Agent Path Finding. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 12353–12362. <https://doi.org/10.1609/aaai.v35i14.17466>
- [5] Hang Ma, Daniel Harabor, Peter J. Stuckey, Jiaoyang Li, and Sven Koenig. 2019. Searching with consistent prioritization for multi-agent path finding. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence (Honolulu, Hawaii, USA) (AAAI'19/IAAI'19/EAAI'19)*. AAAI Press, Article 938, 8 pages. <https://doi.org/10.1609/aaai.v33i01.33017643>
- [6] Keisuke Okumura. 2023. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. AAAI Press, 11655–11662. <https://doi.org/10.1609/aaai.v37i10.26377>
- [7] Alexey Skrynnik, Anton Andreychuk, Anatolii Borzilov, Alexander Chernyavskiy, Konstantin Yakovlev, and Aleksandr Panov. 2025. POGEMA: A Benchmark Platform for Cooperative Multi-Agent Pathfinding. In *The Thirteenth International Conference on Learning Representations*.
- [8] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2024. Decentralized monte carlo tree search for partially observable multi-agent pathfinding. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'24/IAAI'24/EAAI'24)*. AAAI Press, Article 1955, 10 pages. <https://doi.org/10.1609/aaai.v38i16.29703>
- [9] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2024. Decentralized Monte Carlo Tree Search for Partially Observable Multi-Agent Pathfinding. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence (AAAI-24)*. AAAI Press.
- [10] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. Kumar, and Eli Boyarski. 2021. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. *Proceedings of the International Symposium on Combinatorial Search* 10 (09 2021), 151–158. <https://doi.org/10.1609/socs.v10i1.18510>
- [11] Yutong Wang, Xiang Bairan, Shinan Huang, and Guillaume Sartoretti. 2023. SCRIMP: Scalable Communication for Reinforcement- and Imitation-Learning-Based Multi-Agent Pathfinding. 9301–9308. <https://doi.org/10.1109/IROS55552.2023.10342305>